Department of Statistics & Computer Science

University of Kelaniya

COSC 30193/ BECS 31213 – Enterprise Software Design and Architecture

Web Services

Web services are standardized methods that allow different software applications to communicate with each other over the internet, regardless of their underlying technology. They use common protocols like HTTP and data formats such as XML and JSON to enable systems to share data and functionality, making it easier to integrate and interact across various platforms and devices.

Types of Web Services

RESTful Web Services

- Operate on REST principles using standard HTTP methods (GET, POST, PUT, DELETE). Each method corresponds to a common action (read, create, update, delete) in managing data via web services.
- Use simple URL endpoints.
- Commonly use JSON (but can also use XML or other formats).
- Designed for web and mobile applications where simplicity, performance, and scalability are key.

SOAP Web Services

- Operate on the SOAP protocol.
- Use XML for message format.
- Designed for more complex, enterprise-level applications requiring strict security and transaction compliance.

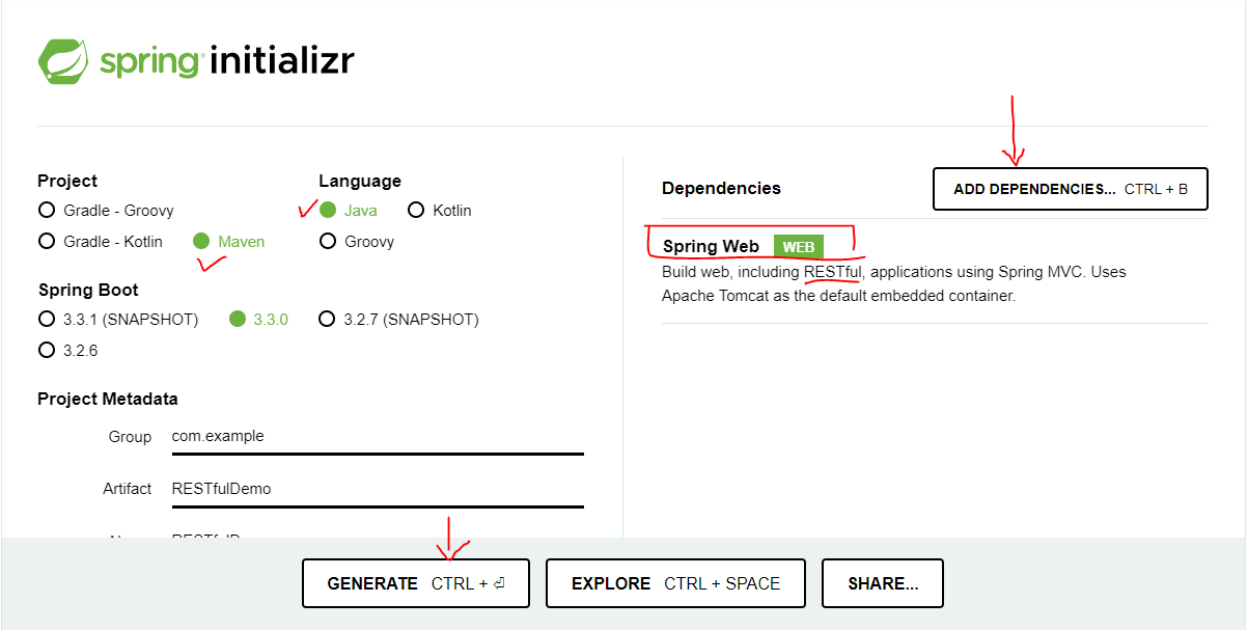
Example for RESTful Web Service

Step 1: Starting with Spring Initializr

To manually initialize the project:

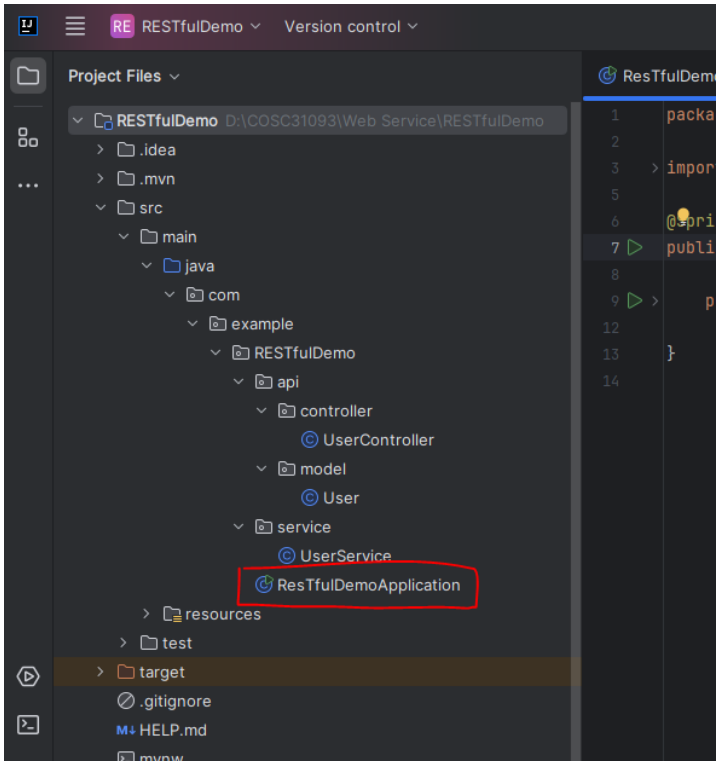
- Navigate to https://start.spring.io. This service pulls in all the dependencies you need for an application and does most of the setup for you.
- Choose either Gradle or Maven and the language you want to use. This guide assumes that you chose Maven and Java.
- Click Dependencies and select Spring Web.
- Click Generate.

- Download the resulting ZIP file, which is an archive of a web application that is configured with your choices.



Step 2: Open the Project Using Intellij Idea

- Create Files and folders as shown in the figure below and create a basic structure.
- The file called ResTfulDemoApplication will be automatically created according to the name given by you.



Step 3: Create User.java Class

```java
package com.example.RESTfulDemo.api.model;

public class User {
    private int id;
    private String name;
    private int age;
    private String email;

   public User(int id,String name, int age, String email){
       this.id=id;
       this.name=name;
       this.age=age;
       this.email=email;
   }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

Step 4: Create UserService.java Class

```java
package com.example.RESTfulDemo.service;

import com.example.RESTfulDemo.api.model.User;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

@Service
public class UserService {

    private List<User> userList;

    public UserService() {
        userList = new ArrayList<>();

        User user1= new User(1,"Nimal",23,"Nimal@gmail.com");
        User user2= new User(2,"Bimal",22,"Bimal@gmail.com");
        User user3= new User(3,"Kamal",21,"Kamal@gmail.com");
        userList.addAll(Arrays.asList(user1,user2,user3));
    }

    public User getUser(Integer id) {
        for(User user:userList){
            if (id==user.getId()){
                return user;
            }
        }
        return null;
    }
}
```

- @Service is a Spring annotation indicating that this class provides some business functionalities. Spring will detect this class during component scanning and create an instance of it as a Spring bean.
- private List<User> userList; declares an instance variable userList which will hold a list of User objects.
- public UserService() is the constructor of the UserService class.
- userList = new ArrayList<>(); initializes userList as an empty ArrayList.
- Three User objects are created with specific ID, name, age, and email.
- userList.addAll(Arrays.asList(user1, user2, user3)); adds these user objects to the userList.
- public User getUser(Integer id) is a method that takes an Integer id as a parameter and returns a User object.

Step 5: Create UserController.java Class

```java
package com.example.RESTfulDemo.api.controller;

import com.example.RESTfulDemo.api.model.User;
import com.example.RESTfulDemo.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;


@RestController
public class UserController {
    private UserService userService;

    @Autowired
    public UserController(UserService userService){
        this.userService=userService;
    }

    @GetMapping("/user")
    public User getUser(@RequestParam Integer id) {
        User user=userService.getUser(id);
        return user;
    }
}
```

- @RestController is a Spring annotation indicating that this class will handle web requests and the return values of its methods will be written directly to the HTTP response body (as JSON or XML, for example).
- private UserService userService; declares an instance variable userService of type UserService.
- @Autowired is a Spring annotation used to inject the UserService bean into the UserController.
- public UserController(UserService userService){ this.userService = userService; } is the constructor for the UserController class. The UserService instance is injected into this constructor by Spring, and it initializes the userService instance variable.
- @GetMapping("/user") maps HTTP GET requests to /user URL to this method.
- public User getUser(@RequestParam Integer id) is a method that handles the GET request and takes an id parameter from the request.
- @RequestParam Integer id binds the id parameter from the HTTP request to this method parameter.
- Inside the method, User user = userService.getUser(id); calls the getUser method of userService to fetch a User object based on the provided id.
- return user; returns the User object, which will be serialized to JSON and sent back in the HTTP response.

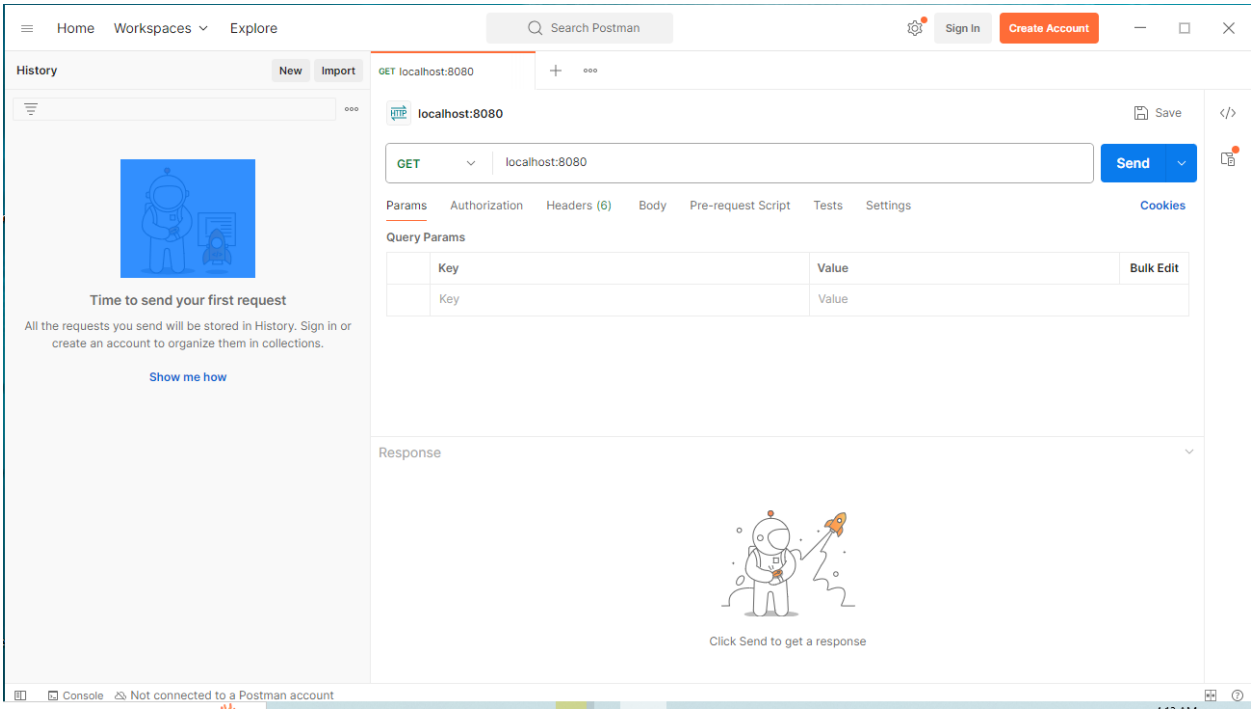
Step 6: Make the Application Executable

- Spring Boot creates an application class (eg: ResTfulDemoApplication.java) for you. In this case, it needs no further modification. You can use it to run this application.

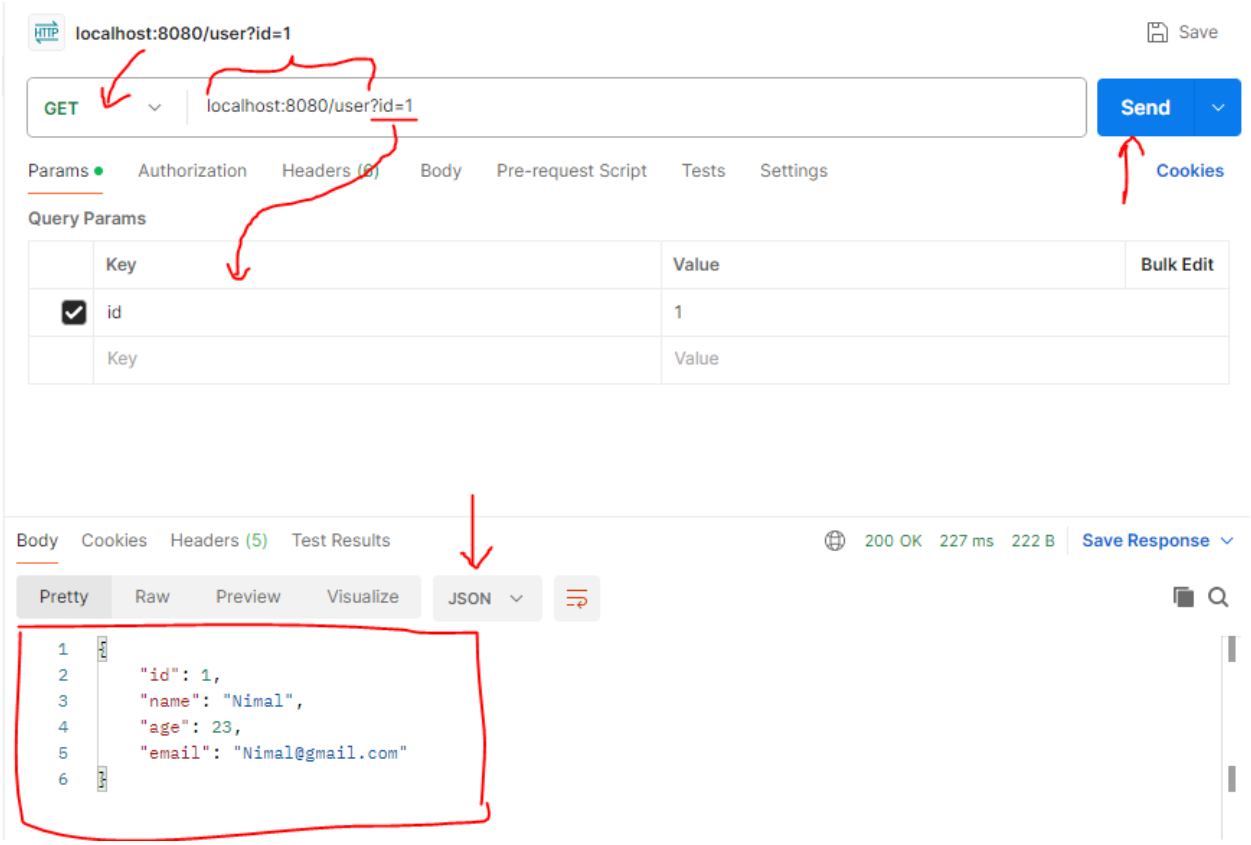
```
package com.example.RESTfulDemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ResTfulDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(ResTfulDemoApplication.class, args);
    }

}
```

Step 7: Test the Application

- The are few options when it comes to testing the RESTful Webservice. You can use something like Postman.
- Postman is a versatile API development and testing tool that allows users to design, test, and document APIs by making HTTP requests and viewing responses in an intuitive interface.



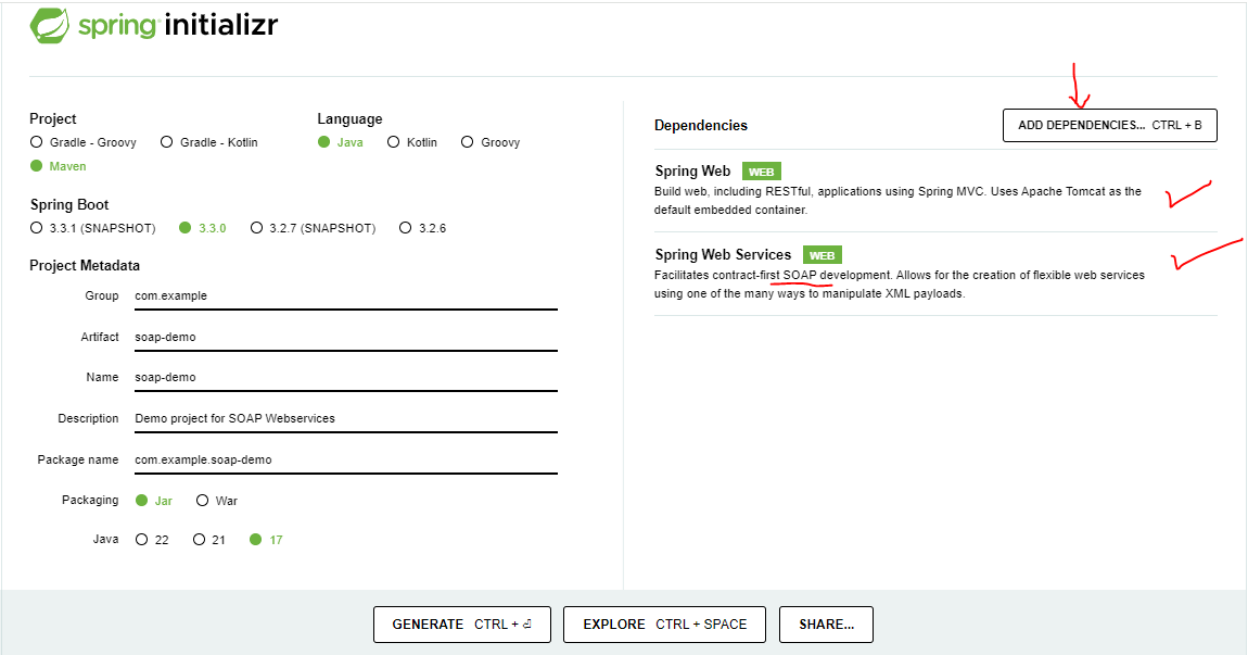
- First run the application and open postman. Set the method to GET and type localhost:8080/user in the text box.
- Next, in params tab, type id under the field key and set the desired user id you want in Value field eg: 1 or 2 or 3.
- Finally click the send button to get the response.



Example for SOAP Web Service

Step 1: Starting with Spring Initializr

- Navigate to https://start.spring.io. This service pulls in all the dependencies you need for an application and does most of the setup for you.
- Choose either Gradle or Maven and the language you want to use. This guide assumes that you chose Maven and Java.
- Click Dependencies and select Spring Web and Spring Web Services.
- Click Generate.
- Download the resulting ZIP file, which is an archive of a web application that is configured with your choices.



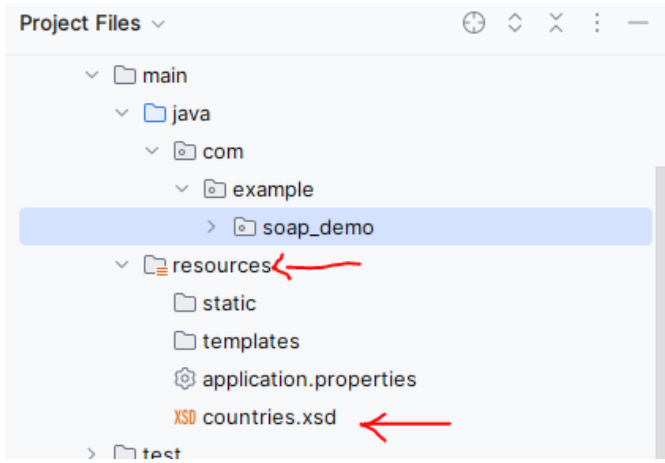
Step 2: Add the Spring-WS dependency

- Open the Project file downloaded from spring initializer in Intellij Idea.
- The project needs to include spring-ws-core and wsdl4j as dependencies in your build file.
- As we use maven, add this dependency to the pom.xml file.

```
<dependency>
    <groupId>wsdl4j</groupId>
    <artifactId>wsdl4j</artifactId>
</dependency>
```

Step 3: Create an XML Schema to Define the Domain

- The web service domain is defined in an XML schema file (XSD) that Spring-WS will automatically export as a WSDL.
- Create an XSD file with operations to return a country's name, population, capital, and currency. The following listing (from src/main/resources/countries.xsd) shows the necessary XSD file:




```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
xmlns:tns="http://spring.io/guides/gs-producing-web-service"
           targetNamespace="http://spring.io/guides/gs-producing-web-service"
elementFormDefault="qualified">

    <xs:element name="getCountryRequest">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="name" type="xs:string"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>

    <xs:element name="getCountryResponse">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="country" type="tns:country"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>

    <xs:complexType name="country">
        <xs:sequence>
            <xs:element name="name" type="xs:string"/>
            <xs:element name="population" type="xs:int"/>
            <xs:element name="capital" type="xs:string"/>
            <xs:element name="currency" type="tns:currency"/>
        </xs:sequence>
    </xs:complexType>
```

```
    <xs:simpleType name="currency">
        <xs:restriction base="xs:string">
            <xs:enumeration value="GBP"/>
            <xs:enumeration value="EUR"/>
            <xs:enumeration value="PLN"/>
        </xs:restriction>
    </xs:simpleType>

</xs:schema>
```

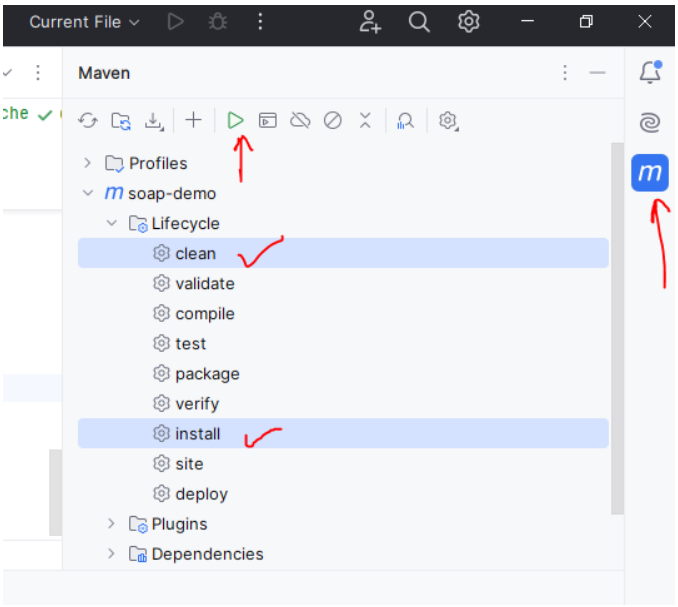
Step 4: Generate Domain Classes Based on an XML Schema

- The next step is to generate Java classes from the XSD file. The right approach is to do this automatically during build time by using a Maven or Gradle plugin.
- As we are focusing on Maven, add the following plugin configuration to pom.xml.
- So, the final pom.xml file will look like this.

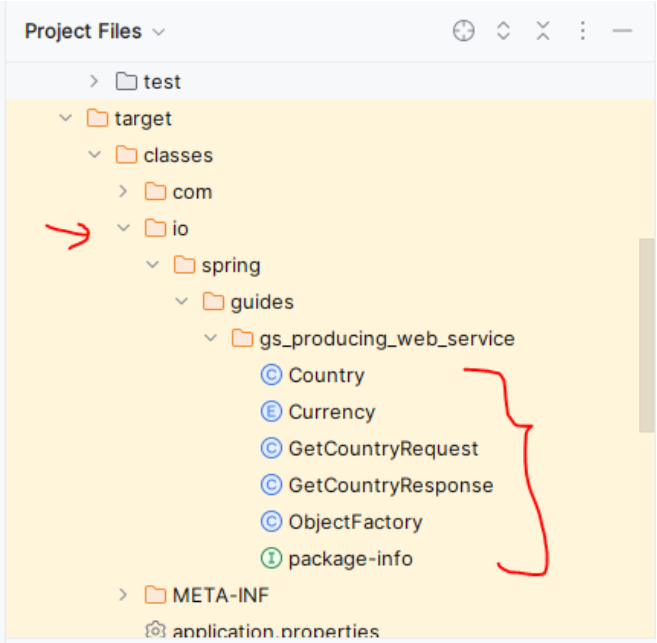
```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.3.0</version>
       <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>soap-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>soap-demo</name>
    <description>Demo project for Spring Boot</description>
    <properties>
       <java.version>17</java.version>
    </properties>
    <dependencies>
       <dependency>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-starter-web</artifactId>
       </dependency>
       <dependency>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-starter-web-services</artifactId>
       </dependency>

       <dependency>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-starter-test</artifactId>
          <scope>test</scope>
       </dependency>
```

```
        <dependency>
            <groupId>wsdl4j</groupId>
            <artifactId>wsdl4j</artifactId>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>jaxb2-maven-plugin</artifactId>
                <version>3.1.0</version>
                <executions>
                    <execution>
                        <id>xjc</id>
                        <goals>
                            <goal>xjc</goal>
                        </goals>
                    </execution>
                </executions>
                <configuration>
                    <sources>

<source>${project.basedir}/src/main/resources/countries.xsd</source>
                    </sources>
                </configuration>
            </plugin>
        </plugins>
    </build>

</project>
```

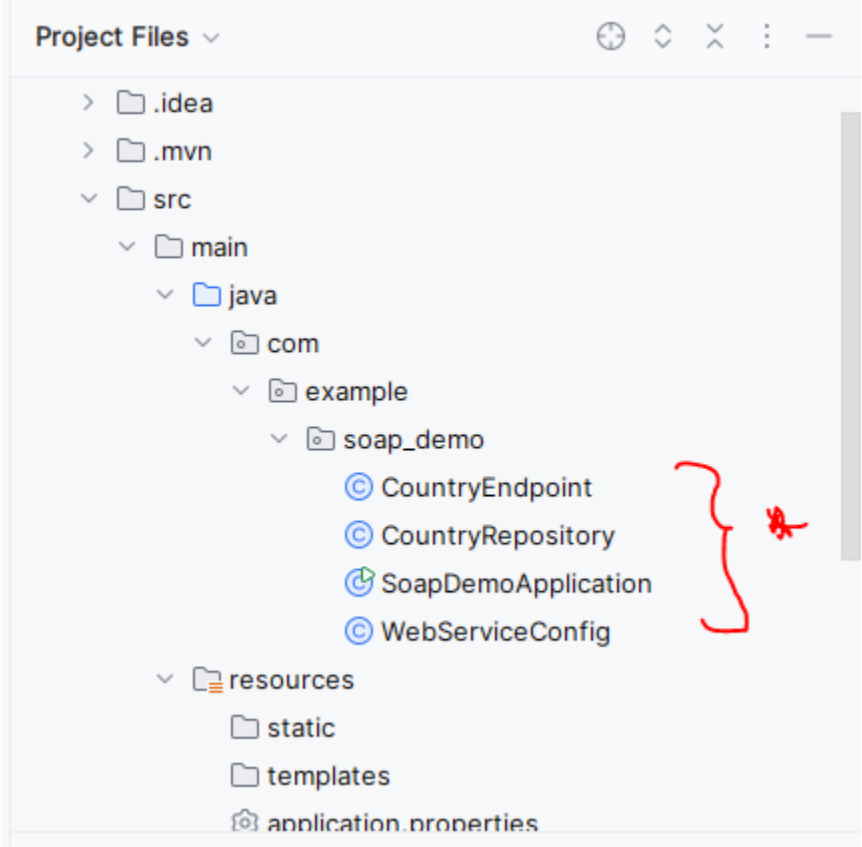
- Go to Maven and run a Clean Install



- This will generate the necessary Java Classes.



- The next step is to create the following files.



Step 5: Create Country Repository

- In order to provide data to the web service, create a country repository. In this guide, you create a dummy country repository implementation with hardcoded data.

```java
package com.example.soap_demo;

import jakarta.annotation.PostConstruct;
import java.util.HashMap;
import java.util.Map;

import io.spring.guides.gs_producing_web_service.Country;
import io.spring.guides.gs_producing_web_service.Currency;
import org.springframework.stereotype.Component;
import org.springframework.util.Assert;

@Component
public class CountryRepository {
    private static final Map<String, Country> countries = new HashMap<>();

    @PostConstruct
    public void initData() {
        Country spain = new Country();
        spain.setName("Spain");
        spain.setCapital("Madrid");
        spain.setCurrency(Currency.EUR);
        spain.setPopulation(46704314);

        countries.put(spain.getName(), spain);

        Country poland = new Country();
        poland.setName("Poland");
        poland.setCapital("Warsaw");
        poland.setCurrency(Currency.PLN);
        poland.setPopulation(38186860);

        countries.put(poland.getName(), poland);

        Country uk = new Country();
        uk.setName("United Kingdom");
        uk.setCapital("London");
        uk.setCurrency(Currency.GBP);
        uk.setPopulation(63705000);

        countries.put(uk.getName(), uk);
    }

    public Country findCountry(String name) {
        Assert.notNull(name, "The country's name must not be null");
        return countries.get(name);
    }
}
```

Step 6: Create Country Service Endpoint

- To create a service endpoint, you need only a POJO with a few Spring WS annotations to handle the incoming SOAP requests.

```
package com.example.soap_demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.ws.server.endpoint.annotation.Endpoint;
import org.springframework.ws.server.endpoint.annotation.PayloadRoot;
import org.springframework.ws.server.endpoint.annotation.RequestPayload;
import org.springframework.ws.server.endpoint.annotation.ResponsePayload;

import io.spring.guides.gs_producing_web_service.GetCountryRequest;
import io.spring.guides.gs_producing_web_service.GetCountryResponse;

@Endpoint

public class CountryEndpoint {
    private static final String NAMESPACE_URI =
"http://spring.io/guides/gs-producing-web-service";

    private CountryRepository countryRepository;

    @Autowired
    public CountryEndpoint(CountryRepository countryRepository) {
        this.countryRepository = countryRepository;
    }

    @PayloadRoot(namespace = NAMESPACE_URI, localPart =
"getCountryRequest")
    @ResponsePayload
    public GetCountryResponse getCountry(@RequestPayload GetCountryRequest
request) {
        GetCountryResponse response = new GetCountryResponse();

response.setCountry(countryRepository.findCountry(request.getName()));

        return response;
    }
}
```

- The @Endpoint annotation registers the class with Spring WS as a potential candidate for processing incoming SOAP messages.

- The @PayloadRoot annotation is then used by Spring WS to pick the handler method, based on the message's namespace and localPart.

- The @RequestPayload annotation indicates that the incoming message will be mapped to the method's request parameter.

- The @ResponsePayload annotation makes Spring WS map the returned value to the response payload.

Step 7: Create Country Service Endpoint

- Create a new class with Spring WS-related beans configuration.

```
package com.example.soap_demo;

import org.springframework.boot.web.servlet.ServletRegistrationBean;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.ws.config.annotation.EnableWs;
import org.springframework.ws.config.annotation.WsConfigurerAdapter;
import org.springframework.ws.transport.http.MessageDispatcherServlet;
import org.springframework.ws.wsdl.wsdl11.DefaultWsdl11Definition;
import org.springframework.xml.xsd.SimpleXsdSchema;
import org.springframework.xml.xsd.XsdSchema;

@EnableWs
@Configuration
public class WebServiceConfig extends WsConfigurerAdapter{
    @Bean
    public ServletRegistrationBean<MessageDispatcherServlet>
messageDispatcherServlet(ApplicationContext applicationContext) {
        MessageDispatcherServlet servlet = new MessageDispatcherServlet();
        servlet.setApplicationContext(applicationContext);
        servlet.setTransformWsdlLocations(true);
        return new ServletRegistrationBean<>(servlet, "/ws/*");
    }

    @Bean(name = "countries")
    public DefaultWsdl11Definition defaultWsdl11Definition(XsdSchema
countriesSchema) {
        DefaultWsdl11Definition wsdl11Definition = new
DefaultWsdl11Definition();
        wsdl11Definition.setPortTypeName("CountriesPort");
        wsdl11Definition.setLocationUri("/ws");
        wsdl11Definition.setTargetNamespace("http://spring.io/guides/gs-
producing-web-service");
        wsdl11Definition.setSchema(countriesSchema);
        return wsdl11Definition;
    }

    @Bean
    public XsdSchema countriesSchema() {
        return new SimpleXsdSchema(new
ClassPathResource("countries.xsd"));
    }
}
```

- Spring WS uses a different servlet type for handling SOAP messages: MessageDispatcherServlet. It is important to inject and set ApplicationContext to MessageDispatcherServlet. Without that, Spring WS will not automatically detect Spring beans.

- Naming this bean messageDispatcherServlet does not replace Spring Boot's default DispatcherServlet bean.

- DefaultMethodEndpointAdapter configures the annotation-driven Spring WS programming model. This makes it possible to use the various annotations, such as @Endpoint (mentioned earlier).

- DefaultWsdl11Definition exposes a standard WSDL 1.1 by using XsdSchema.

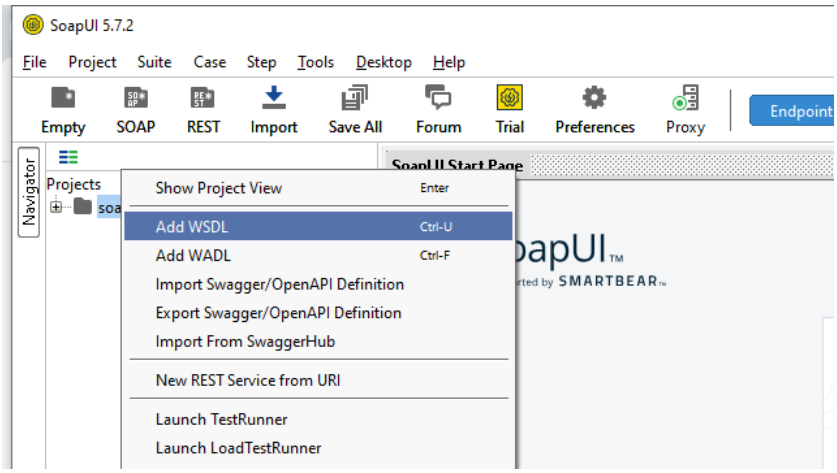
Step 8: Make the Application Executable

- Spring Boot creates an application class (eg: SoapDemoApplication.java) for you. In this case, it needs no further modification. You can use it to run this application.

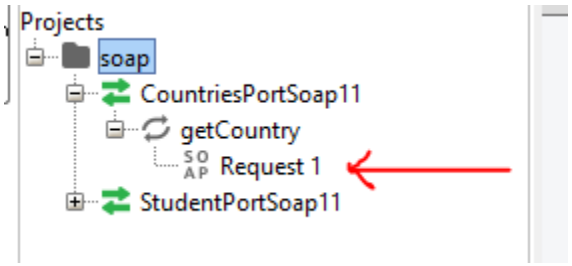
```
package com.example.soap_demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SoapDemoApplication {

    public static void main(String[] args) {
       SpringApplication.run(SoapDemoApplication.class, args);
    }

}
```

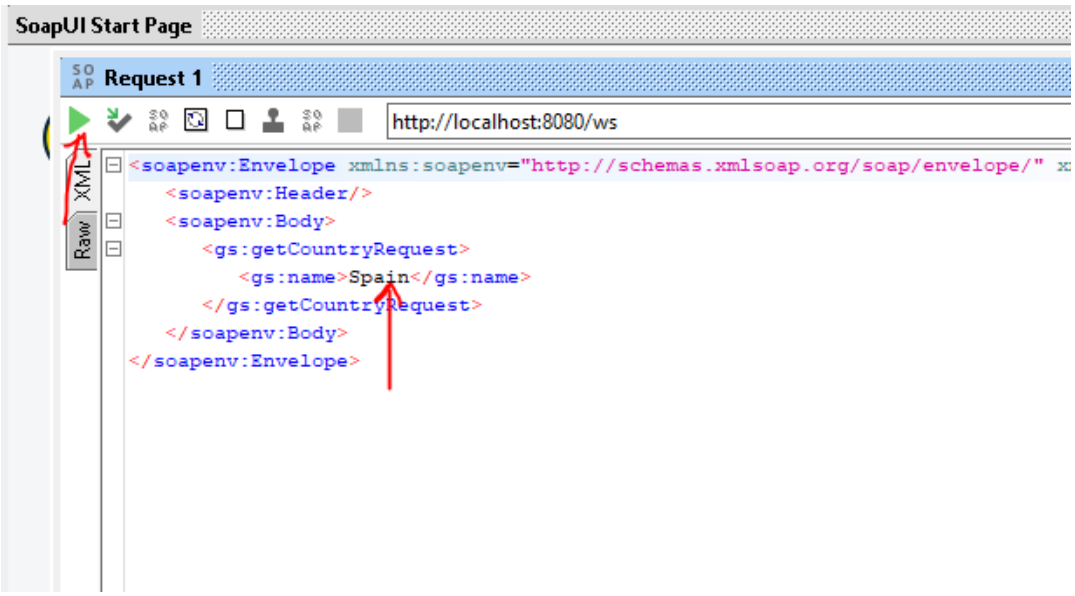
Step 9: Test the Application

- The are few options when it comes to testing the SOAP Webservice. You can use something like SoapUI.
- SoapUI is a cross-platform functional testing tool for SOAP and REST web services, allowing users to perform automated testing, load testing, and compliance testing.
- Access http://localhost:8080/ws/countries.wsdl in your browser and it will show the WSDL (Web Services Description Language) document for your web service.
- Right Click on it and save it as a wsdl file.
- Next Open SoapUI and open the wsdl file.



- After adding the wsdl file follow the following steps





- Finally get the response XML.

http://localhost:8080/ws

XML Raw

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
   <SOAP-ENV:Header/>
   <SOAP-ENV:Body>
      <ns2:getCountryResponse xmlns:ns2="http://spring.io/guides/gs-producing-web-service">
         <ns2:country>
            <ns2:name>Spain</ns2:name>
            <ns2:population>46704314</ns2:population>
            <ns2:capital>Madrid</ns2:capital>
            <ns2:currency>EUR</ns2:currency>
         </ns2:country>
      </ns2:getCountryResponse>
   </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```